SCALER School of Technology · Priyansh Saxena

ML System Design Case Study

Part 2 of 2 ▪ Real-time Recommendation System

Priyansh Saxena

Scaler School of Technology | ML System Design and LLMOps

Week 5 — Session 1/2 (Part 2) /

The Problem Brief

Design a real-time music recommendation system

for a streaming service with **50 million active users**.

Requirements:

- Personalised recommendations
- p99 latency < 100 ms
- 10,000 requests per second at peak
- New songs visible within 24 hours
- Cold-start handling for new users

We will now apply all 10 rubric steps to this problem.

1–3 Clarify, Context, Problem Statement

Clarifying questions & answers

- Catalogue size? *100M songs*
- Personalisation granularity? *Per-user, updated nightly*
- Availability SLA? *99.9%*
- Scope? *ML system only, not the app*

Context: Music streaming competes on engagement and discovery. Personalised recommendations drive session length, retention, and artist discovery.

Problem Statement

Given a user's interaction history and current session context, rank a candidate set by probability of play and return the top 20.

Input: user ID + session signals
Output: ranked list of 20 song IDs
Constraint: $\leq$100 ms end-to-end

4 Requirements

Functional

- Personalised ranking per user
- New songs indexed within 24 hours
- Cold-start: useful recs with 0 history
- Feedback: plays, skips, likes

Non-functional

- p99 latency < 100 ms
- 10,000 RPS at peak
- 99.9% availability
- Stateless pods (horizontal scaling)
- Graceful degradation if cache fails

Key implication

100 ms p99 + **100M songs** = you *cannot* rank all songs at serving time. This single constraint determines the entire architecture.

5 Estimations

What	Estimate	Implication
Daily events	∼500M plays/skips	250 GB/day of event data
User features	50M × 128 dims	≈25 GB in Redis
Song ANN index	100M × 128 dims	≈50 GB FAISS index
Serving compute	10K RPS × 90 ms	≈900 concurrent reqs
Training compute	15B pairs/night	4–8 GPU hours nightly

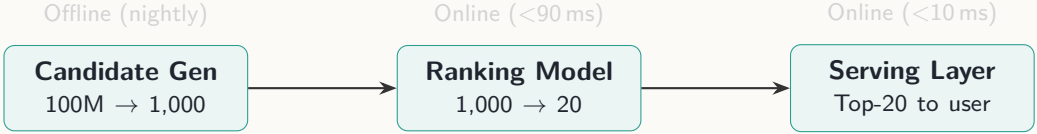
What these numbers tell us

50 GB ANN index fits in memory on one machine. 250 GB/day of events means nightly batch is sufficient — streaming is not needed for the 24h SLA.

6 Architecture — The Key Insight

You cannot rank 100 million songs in real-time under 100 ms.

So you don't. **Pre-filter offline** to ∼1,000 candidates per user, then **rank online** in real-time.



Every major recommender uses this pattern: Spotify, Netflix, YouTube, TikTok, Amazon.

6 Architecture — Components

(a) Data Collection
Kafka streams: plays, skips, likes, searches. Log *all* events — skips are signal too.

(b–c) Data Engineering & Feature Store
Nightly Spark → S3 (offline, point-in-time correct).
Redis (online, <5 ms lookup, 4h refresh).

(d) Candidate Generation
Two-tower model → FAISS ANN → 1,000 candidates cached in Redis.

(e) Models
Cand. gen: Two-tower NN on implicit feedback (plays).
Ranker: LightGBM on user-song pairs.
MLflow Registry versions both independently.

(f) Post-processing
Deduplicate artists. Inject 1 novel artist per 5. Apply business rules (geo, licensing). Return top 20.

6 Architecture — Latency Budget

#	Operation	Budget (ms)
1	Network + load balancer	5
2	Fetch candidate list from Redis	5
3	Fetch user context features from Redis	5
4	Run ranker on 1,000 candidates (Triton)	75
5	Serialize + network	10
	Total p99 target	**100 ms**

Budget explicitly, not aspirationally

Each step has an allowance. If step 4 takes 90 ms, steps 1–3 must fit in 10 ms combined. Design to the budget, then measure it.

7 Evaluation

Offline

- Precision@20 / Recall@20
- NDCG@20 (ranking quality)
- AUC-ROC (click prediction)

Online

- **North star:** CTR (7-day rolling)
- Session length, artist discovery rate
- **Guardrail:** Diversity score
  must not drop $>10\%$ week-over-week

Why diversity as a guardrail?

Recommending the same 100 popular songs to everyone raises CTR short-term but destroys long-term retention. Without a diversity guardrail the optimiser will exploit this.

8 A/B Testing

Experiment design

- **Unit:** User ID (not session — avoids novelty contamination)
- **Split:** 10% treatment → 50/50 after 48h
- **Duration:** 14 days minimum
- **Sample:** 5M users detects 1% CTR lift at α=0.05

Decision rule

Promote if: CTR $\uparrow \geq 0.5\%$ **and** diversity does not drop $>5\%$

Rollback if: p99 >120 ms or error rate $\uparrow > 0.1\%$ within 24h

Bias check

Verify treatment/control distributions match on age, device, and listening frequency before starting.

9 Deployment and Monitoring

Deployment

- Airflow DAGs: nightly training + 4h candidate refresh
- MLflow gate: must beat production NDCG@20 before promotion
- Blue-green: 10% shadow traffic first
- One-command rollback via Registry

6 Metrics to Monitor

1. p99 latency — alert if >120 ms
2. Feature store freshness lag
3. Diversity score
4. CTR — alert on $>5\%$ drop
5. Cold-start vs. warm CTR gap
6. Candidate list staleness

10 Risk Management and Trade-offs

Decision	Choice	Alternative	Why
Retrieval	Two-stage	Single ranker	Can't rank 100M in $<$100 ms
Candidate freshness	4h pre-compute	Real-time ANN	Real-time ANN at 10K RPS too costly
Ranker	LightGBM	Neural net	Faster, interpretable, comparable accuracy
Training signal	Plays (implicit)	Likes (explicit)	Plays 100$\times$ more abundant
Cold-start	Tiered fallback	None	User with nothing to show churns immediately

Ethical risk: popularity bias

Without diversity guardrail the ranker recommends popular songs aggressively. New artists get no exposure. CTR up, long-term retention down.

Cold-Start: Product Problem, Not ML Problem

Key framing

Cold-start is a **product problem masquerading as an ML problem**. The fix is a tiered fallback — not a better model.

User state	Strategy	Data source
0 interactions	Global popularity + editorial	Aggregated play counts
1–4 interactions	Content-based	Audio features (genre, BPM, mood)
5–19 interactions	Collab + content hybrid	Partial history + song embeddings
20+ interactions	Full two-tower	Full interaction history

Failure Modes and Recovery

What breaks	How to detect	Recovery
Redis cache fails	Latency spike; hit rate $= 0$	Fallback to global popularity; rebuild from offline store
Nightly training fails	MLflow: no new run; staleness alert	Serve previous model; trigger manual retraining
Feature skew	Evidently drift; CTR drops, latency unchanged	Pause promotion; compare distributions; fix pipeline
Model degrades	CTR $>5\%$ drop week-over-week	Roll back via Registry; investigate training data
Cold-start loop	Cold-start CTR $\gg$ warm; no transition	Check interaction logging; verify feature store updates

What We Covered Today

Part 1 recap — The Rubric

- 10-step framework for any ML design question
- Steps 1–5: frame the problem first
- Step 6: architecture sub-steps (a)–(f)
- Steps 7–10: validate and operate

Part 2 — This Case Study

- Two-stage architecture: the key insight
- Latency budget: explicit, not aspirational
- Diversity as a guardrail
- Cold-start: tiered product solution
- Failure modes per component

Three takeaways

1. **Scale forces architecture.** 100 ms + 100M songs = two-stage, always.
2. **Every component has a failure mode.** Design for it upfront.
3. **The rubric works.** Same 10 questions, any problem.

Good luck applying the rubric!

Every ML system design question has the same ten questions hidden inside it.



Priyansh Saxena

https://www.priyanshsaxena.com/sst-ml-system-design